Etude de cas 3 :

Kanban

Mathéo Pohier

Exercice 1 — MVP

Hypothèse de départ

L’entreprise Jaydee a des problèmes pour suivre l’avancement de sa production, donc elle a besoin d’un outil simple pour visualiser les tâches.

Cible

- Les chefs d’atelier
- Les opérateurs
- Les responsables de production

Problème utilisateur

Aujourd’hui :

- On ne voit pas bien où en sont les tâches
- Les priorités changent souvent
- Il y a des retards

Proposition de valeur

Créer une application web type Kanban qui permet de :

- voir toutes les tâches facilement
- organiser le travail par colonnes
- améliorer la gestion globale

Fonctionnalités du MVP

- Affichage d’un tableau Kanban
- Colonnes (A faire, En cours, Terminé…)
- Tâches avec une couleur
- Ajout de tâche
- API pour récupérer les données

Fonctionnalités exclues

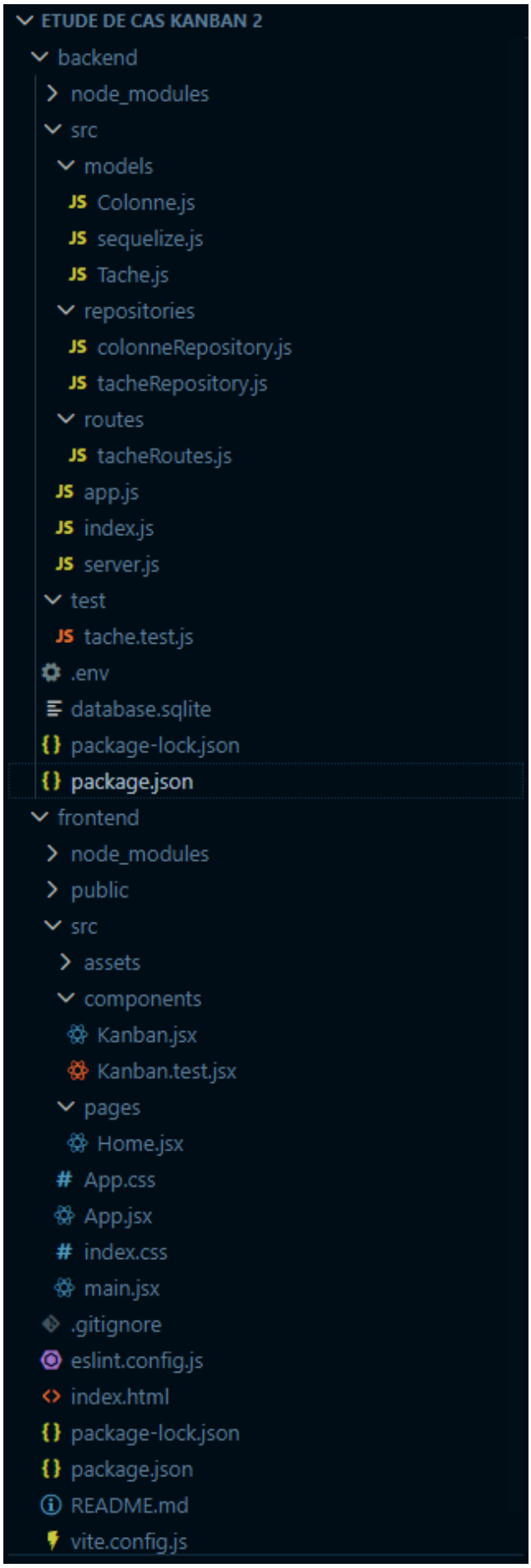
- connexion utilisateur
- drag & drop
- modification/suppression
- notifications

Exemple du Wireframe :

Exercice 2 – Initialisation

Voici en capture d'écran les étapes pour l'initialisation du projet Kanban

L'arborescence


ETUDE DE CAS KANBAN 2
backend
node_modules
src
models
Colonne.js
sequelize.js
Tache.js
repositories
colonneRepository.js
tacheRepository.js
routes
tacheRoutes.js
app.js
index.js
server.js
test
tache.test.js
.env
database.sqlite
package-lock.json
package.json
frontend
node_modules
public
src
assets
components
Kanban.jsx
Kanban.test.jsx
pages
Home.jsx
App.css
App.jsx
index.css
main.jsx
.gitignore
eslint.config.js
index.html
package-lock.json
package.json
README.md
vite.config.js

L’installation de Vite :

```
Mathéo@DESKTOP-7PHC6FL MINGW64 /j/COURS ISCOD/Etude de cas Kanban 2
$ npm create vite@latest frontend -- --template react
Need to install the following packages:
create-vite@9.0.3
Ok to proceed? (y)

> npx
> create-vite frontend --template react


◇  Target directory "frontend" is not empty. Please choose how to proceed:
   Ignore files and continue

◇  Install with npm and start now?
   Yes

◇  Scaffolding project in J:\COURS ISCOD\Etude de cas Kanban 2\frontend...

◇  Installing dependencies with npm...

added 151 packages, and audited 152 packages in 10s

36 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

◇  Starting dev server...

> frontend@0.0.0 dev
> vite


  VITE v8.0.3  ready in 602 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
```

Package.json du frontend :

frontend > {} package.json > { } devDependencies

```
1   {
2     "name": "frontend",
3     "private": true,
4     "version": "0.0.0",
5     "type": "module",
      ▷Debug
6     "scripts": {
7       "dev": "vite",
8       "build": "vite build",
9       "lint": "eslint .",
10      "preview": "vite preview",
11      "test": "jest"
12    },
13    "dependencies": {
14      "axios": "^1.13.6",
15      "react": "^19.2.4",
16      "react-dom": "^19.2.4"
17    },
18    "devDependencies": {
19      "@eslint/js": "^9.39.4",
20      "@testing-library/jest-dom": "^6.9.1",
21      "@testing-library/react": "^16.3.2",
22      "@types/react": "^19.2.14",
23      "@types/react-dom": "^19.2.3",
24      "@vitejs/plugin-react": "^6.0.1",
25      "eslint": "^9.39.4",
26      "eslint-plugin-react-hooks": "^7.0.1",
27      "eslint-plugin-react-refresh": "^0.5.2",
28      "globals": "^17.4.0",
29      "jest": "^30.3.0",
30      "supertest": "^7.2.2",
31      "vite": "^8.0.1"
32    }
33  }
34
```

Package.json du backend :

```
backend > {} package.json > ...
1  {
2    "name": "backend",
3    "version": "1.0.0",
4    "main": "src/server.js",
       ▷Debug
5    "scripts": {
6      "dev": "nodemon src/server.js",
7      "test": "jest"
8    },
9    "dependencies": {
10     "dotenv": "^17.3.1",
11     "express": "^4.18.2",
12     "sequelize": "^6.37.8",
13     "sqlite3": "^6.0.1"
14   },
15   "devDependencies": {
16     "jest": "^29.7.0",
17     "nodemon": "^3.1.14",
18     "supertest": "^6.3.4"
19   }
20 }
21
```

Exercice 3 – Vérification

Vérification Frontend :

```
Mathéo@DESKTOP-7PHC6FL MINGW64 /j/COURS ISCOD/Etude de cas Kanban 2/frontend
$ npm run dev

> frontend@0.0.0 dev
> vite


  VITE v8.0.3  ready in 169 ms

  ➜ Local:   http://localhost:5173/
  ➜ Network: use --host to expose
  ➜ press h + enter to show help
```

Vérification Backend :

```
Mathéo@DESKTOP-7PHC6FL MINGW64 /j/COURS ISCOD/Etude de cas Kanban 2/backend
$ npm run dev

> backend@1.0.0 dev
> nodemon src/server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node src/server.js`
[dotenv@17.3.1] injecting env (1) from .env -- tip: 🛠 run anywhere with `dotenvx run -- yourcommand`
Server running on http://localhost:3000
```

Le Health pour la vérification :

```
JS server.js ×   .env   {} package.json
backend > src > JS server.js > ...
1   require('dotenv').config();
2   const express = require('express');
3
4   const app = express();
5
6   const PORT = process.env.PORT || 3000;
7
8   app.get('/health', (req, res) => {
9     res.json({ status: 'OK' });
10  });
11
12  app.listen(PORT, () => {
13    console.log(`Server running on http://localhost:${PORT}`);
14  });
```

Le test Curl :

```
Mathéo@DESKTOP-7PHC6FL MINGW64 /j/COURS ISCOD/Etude de cas Kanban 2/backend
$ curl http://localhost:3000/health
{"status":"OK"}
```

Exercice 4 – Modèles

Modèles Colonnes :

backend > src > models > JS Colonne.js > ...

```javascript
const { DataTypes } = require('sequelize');
const sequelize = require('./sequelize');

const Colonne = sequelize.define('Colonne', {
    id: {
      type: DataTypes.BIGINT,
      primaryKey: true,
      autoIncrement: true
    },
    intitule: {
      type: DataTypes.STRING,
      allowNull: false
    }
  });

module.exports = Colonne;
```

Modèles Tache :

backend > src > models > JS Tache.js > Tache > couleur > validate

```javascript
const { DataTypes } = require('sequelize');
const sequelize = require('./sequelize');
const Colonne = require('./Colonne');

const Tache = sequelize.define('Tache', {
    id: {
      type: DataTypes.BIGINT,
      primaryKey: true,
      autoIncrement: true
    },
    nom: {
      type: DataTypes.STRING,
      allowNull: false,
      validate: {
        notEmpty: true
      }
    },
    couleur: {
      type: DataTypes.STRING,
      allowNull: false,
      validate: {
        is: /^[0-9A-Fa-f]{6}$/
      }
    }
  });

Tache.belongsTo(Colonne);
Colonne.hasMany(Tache);

module.exports = Tache;
```

Exercice 5 – SQL

```
MariaDB [kanban]> create table colone (
    -> id bigint primary key auto_increment,
    -> intitule varchar(255) not null
    -> );
```

```
MariaDB [kanban]> create table tache (
    -> id bigint primary key auto_increment,
    -> nom varchar(255) not null,
    -> couleur varchar(6) not null,
    -> colonne_id bigint,
    -> foreign key (colonne_id) references colone(id)
    -> );
Query OK, 0 rows affected (0.034 sec)
```

Exercice 6 – Repository

Colonne :

```
JS server.js    JS Colonne.js    JS Tache.js    JS colonneRepository.js
backend > src > repositories > JS colonneRepository.js > ...
1  const { Colonne } = require('backend/src/models');
2
3  exports.findAll = () => Colonne.findAll();
4  exports.create = (data) => Colonne.create(data);
```

Tache :

```
JS server.js    JS Colonne.js    JS Tache.js    JS colonneRepository.js    JS tacheRepository.js
backend > src > repositories > JS tacheRepository.js > ...
1  const { Tache, Colonne } = require('backend/src/models');
2
3  exports.findAll = () => Tache.findAll({ include: Colonne });
4  exports.create = (data) => Tache.create(data);
```

Exercice 7 – Données

```
MariaDB [kanban]> insert into colone (intitule) values
    -> ('a faire'),
    -> ('en cours'),
    -> ('a tester'),
    -> ('terminé');
Query OK, 4 rows affected, 1 warning (0.024 sec)
Records: 4  Duplicates: 0  Warnings: 1
```

```
MariaDB [kanban]> INSERT INTO tache (nom, couleur, colonne_id) VALUES('Tâche 1', 'FF0000', 1),('Tâche 2', '00FF00', 1),('Tâche 3', '0000FF',
 2),('Tâche 4', 'FFFF00', 3),('Tâche 5', 'FF00FF', 4);
Query OK, 5 rows affected, 5 warnings (0.007 sec)
Records: 5  Duplicates: 0  Warnings: 5
```

Exercice 8 – Affichage

Backend :

```
app.get('/api/taches', (req, res) => {
  res.json([
    { id: 1, nom: "Tâche 1", couleur: "FF0000" },
    { id: 2, nom: "Tâche 2", couleur: "00FF00" }
  ]);
});
```

React :

```
frontend > src > App.jsx > App
1   import { useEffect, useState } from 'react';
2   import axios from 'axios';
3
4   function App() {
5     const [taches, setTaches] = useState([]);
6
7     useEffect(() => {
8       axios.get('http://localhost:3000/api/taches')
9         .then(res => setTaches(res.data));
10    }, []);
11
12    return (
13      <div>
14        <h1>Kanban</h1>
15
16        {taches.map(t => (
17          <div key={t.id}>
18            {t.nom}
19          </div>
20        ))}
21      </div>
22    );
23  }
24
25  export default App;
```

Test :

```
Mathéo@DESKTOP-7PHC6FL MINGW64 /j/COURS ISCOD/Etude de cas Kanban 2/backend
$ curl http://localhost:3000/api/taches
[{"id":1,"nom":"Tâche 1","couleur":"FF0000"},{"id":2,"nom":"Tâche 2","couleur":"00FF00"}]
```

Exercice 9 – POST

Backend :

backend > src > routes > JS tacheRoutes.js > ...

```javascript
const express = require('express');
const router = express.Router();
const Tache = require('../models/Tache');
const Colonne = require('../models/Colonne');

router.get('/', (req, res) => {
  res.json([]);
});

router.post('/', async (req, res) => {
  try {
    const colonne = await Colonne.findOne({ where: { intitule: 'A Faire' } });
    if (!colonne) {
      return res.status(400).json({ error: 'Colonne A Faire inexistante' });
    }

    const tache = await Tache.create({
      nom: req.body.nom,
      couleur: req.body.couleur,
      colonneId: colonne.id
    });

    res.status(201).json(tache);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

module.exports = router;
```

backend > src > JS server.js > app.get('/api/taches') callback

```javascript
require('dotenv').config();
const express = require('express');

const app = express();

const PORT = process.env.PORT || 3000;

const tacheRoutes = require('./routes/tacheRoutes');

app.use(express.json());
app.use('/api/taches', tacheRoutes);

app.get('/health', (req, res) => {
  res.json({ status: 'OK' });
});

app.get('/api/taches', (req, res) => {
  res.json([
    { id: 1, nom: "Tâche 1", couleur: "FF0000" },
    { id: 2, nom: "Tâche 2", couleur: "00FF00" }
  ]);
});

module.exports = app;
```

Test :

backend > test > JS tache.test.js > beforeAll() callback

```javascript
const request = require('supertest');
const app = require('../src/app');
const Tache = require('../src/models/Tache');
const Colonne = require('../src/models/Colonne');
const sequelize = require('../src/models/sequelize');

beforeAll(async () => {
  await sequelize.sync({force:true});
  await Colonne.sync({ force: true });
  await Tache.sync({ force: true });
  await Colonne.create({ intitule: 'A Faire' });
});

afterAll(async () => {
  const sequelize = require('../src/models/Tache').sequelize;
  await sequelize.close();
});

describe('Test API Taches', () => {

  it('GET /api/taches doit renvoyer 200 et un tableau', async () => {
    const res = await request(app).get('/api/taches');

    expect(res.statusCode).toBe(200);
    expect(Array.isArray(res.body)).toBe(true);
  });

  it('POST /api/taches doit ajouter une tâche', async () => {
    const nouvelleTache = {
      nom: 'Test',
      couleur: 'FF0000'
    };

    const res = await request(app)
      .post('/api/taches')
      .send(nouvelleTache);

    expect(res.statusCode).toBe(201);
    expect(res.body.nom).toBe('Test');
  });

});
```

```
PASS  test/tache.test.js
  Test API Taches
    √ GET /api/taches doit renvoyer 200 et un tableau (13 ms)
    √ POST /api/taches doit ajouter une tâche (95 ms)

Test Suites: 1 passed, 1 total
Tests:       2 passed, 2 total
Snapshots:   0 total
Time:        1.841 s, estimated 2 s
Ran all test suites.
```

Conclusion :

Voici comment j’ai mis en place une API Node + un front React pour afficher un Kanban simple.

Le plus important c’est :

- bien structurer le projet
- respecter les couches
- gérer les relations en base
- faire des tests basiques